CPS Report

AI-Based Intrusion Detection System for IoT Networks: Detecting DoS, MITM, and Data Injection Attacks

Arab Academy for Science, Technology & Maritime Transport
College of Computing & Information Technology

Course: Cyber-Physical Systems (CPS)

Prepared by:

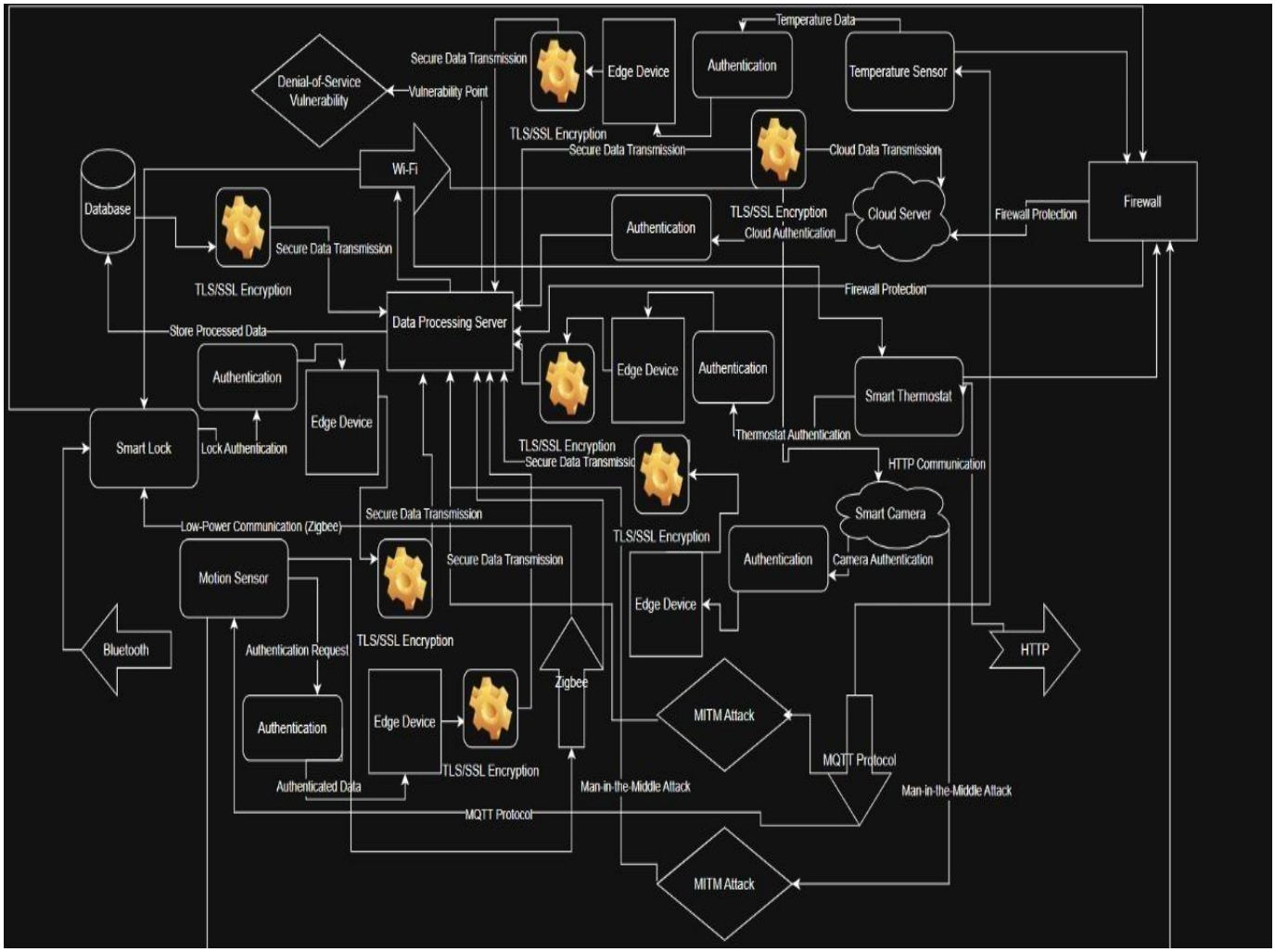
- **Nour Mohamed** **221027637**
- **Mohamed Hesham** **221008918**
- **Abdelrahman Mohamed** **221027770**
- **Youssef Khaled** **221027758**

1. Introduction

This project presents the design and implementation of an AI-Based Intrusion Detection System (IDS) for IoT and Cyber-Physical Systems (CPS), aiming to address the growing security challenges caused by large-scale connectivity, heterogeneous devices, and limited computational resources. Traditional rule-based IDS solutions are often ineffective in IoT environments due to their inability to adapt to evolving and unknown attack patterns. To overcome these limitations, the proposed system leverages machine learning techniques—specifically a Random Forest classifier—trained on network traffic features to distinguish between normal and malicious behavior. The IDS is capable of detecting common IoT-related attacks such as Denial of Service (DoS), Man-in-the-Middle (MITM), and Data Injection attacks through realistic traffic simulations. A Flask-based web interface is integrated to provide real-time attack simulation, detection feedback, and monitoring, making the system both interactive and practical for demonstration. Experimental results show that the system can detect approximately 45–50% of malicious packets in real-time traffic streams, highlighting both its effectiveness and current limitations, while providing a solid foundation for future optimization and deployment in resourceconstrained IoT environments.

2. System Architecture Overview

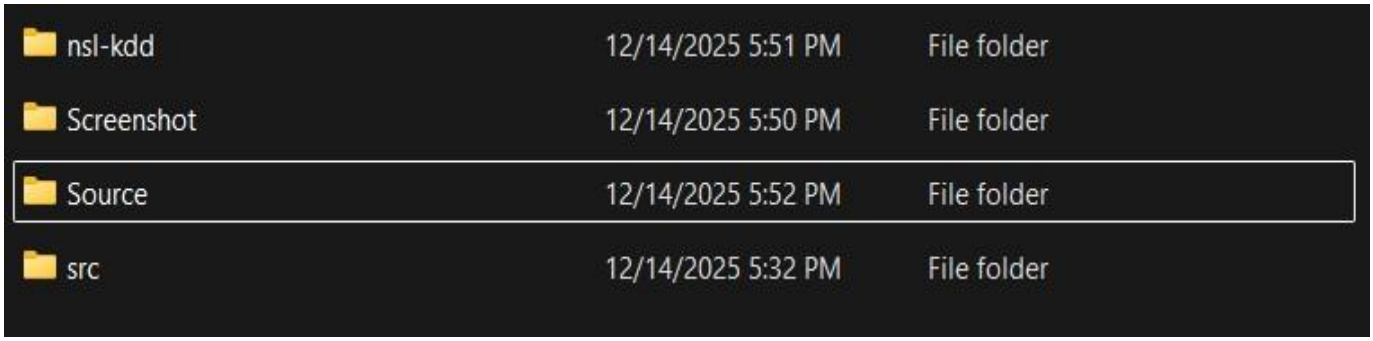
The system architecture of the proposed AI-based IoT Intrusion Detection System is designed as a layered and distributed framework that reflects real-world IoT and cyber-physical environments. It consists of multiple IoT devices and sensors (such as smart locks, cameras, thermostats, and motion sensors) that communicate using heterogeneous protocols including Wi-Fi, Zigbee, Bluetooth, HTTP, and MQTT. Data generated by these devices is securely transmitted through edge devices and gateways using TLS/SSL encryption and authentication mechanisms before reaching the data processing server. This server is responsible for traffic analysis and hosts the machine learning–based IDS, which continuously monitors network packets to detect malicious activities such as DoS, MITM, and data injection attacks. Detected events and processed data are stored in a centralized database and optionally forwarded to cloud services protected by firewall mechanisms. The architecture emphasizes secure communication, distributed processing, and real-time detection, ensuring scalability and suitability for resource-constrained IoT environments while maintaining effective intrusion monitoring.



This system architecture diagram clearly illustrates the security boundaries within the proposed AI-based IoT Intrusion Detection System to highlight trust relationships and potential attack surfaces. The **In-Device / Local IoT Network Boundary** separates internal IoT components—such as smart locks, cameras, thermostats, and sensors—from external entities, emphasizing secure local communications via protocols like Zigbee, Bluetooth, MQTT, and HTTP. The **Edge–Cloud Trust Boundary** defines the transition point where data moves from edge devices and the data processing server to cloud services, enforcing TLS/SSL encryption, authentication, and firewall protection to prevent data leakage and unauthorized access. Finally, the **External Environment Boundary** represents untrusted networks and attackers, where threats such as Denial-of-Service (DoS) and Man-in-the-Middle (MITM) attacks may originate. By clearly defining these boundaries, the diagram exposes trust violations, communication weaknesses, and attack vectors, enabling the IDS to monitor traffic across boundaries and detect malicious activities before they compromise the system.

3. install dependencies and the requirments

This step demonstrates the setup of a clean and isolated Python development environment for the AI-Based IoT Intrusion Detection System. First, a **virtual environment** is created using the command python -m venv env, which ensures that all project dependencies are installed separately from the system-wide Python installation, preventing version conflicts. The environment is then activated in PowerShell using .\env\Scripts\Activate.ps1, indicated by the (env) prefix in the terminal, confirming that all subsequent commands run inside the virtual environment. After activation, the required libraries are installed using pip install, including **NumPy and Pandas** for data handling, **Scikit-learn** for machine learning model training, **TensorFlow and Keras** for advanced learning support, **Flask** for building the web-based IDS interface, **Matplotlib** for visualization, and **Joblib** for saving and loading trained models. This step ensures that the project has all necessary dependencies correctly installed and ready for training, simulation, and deployment.



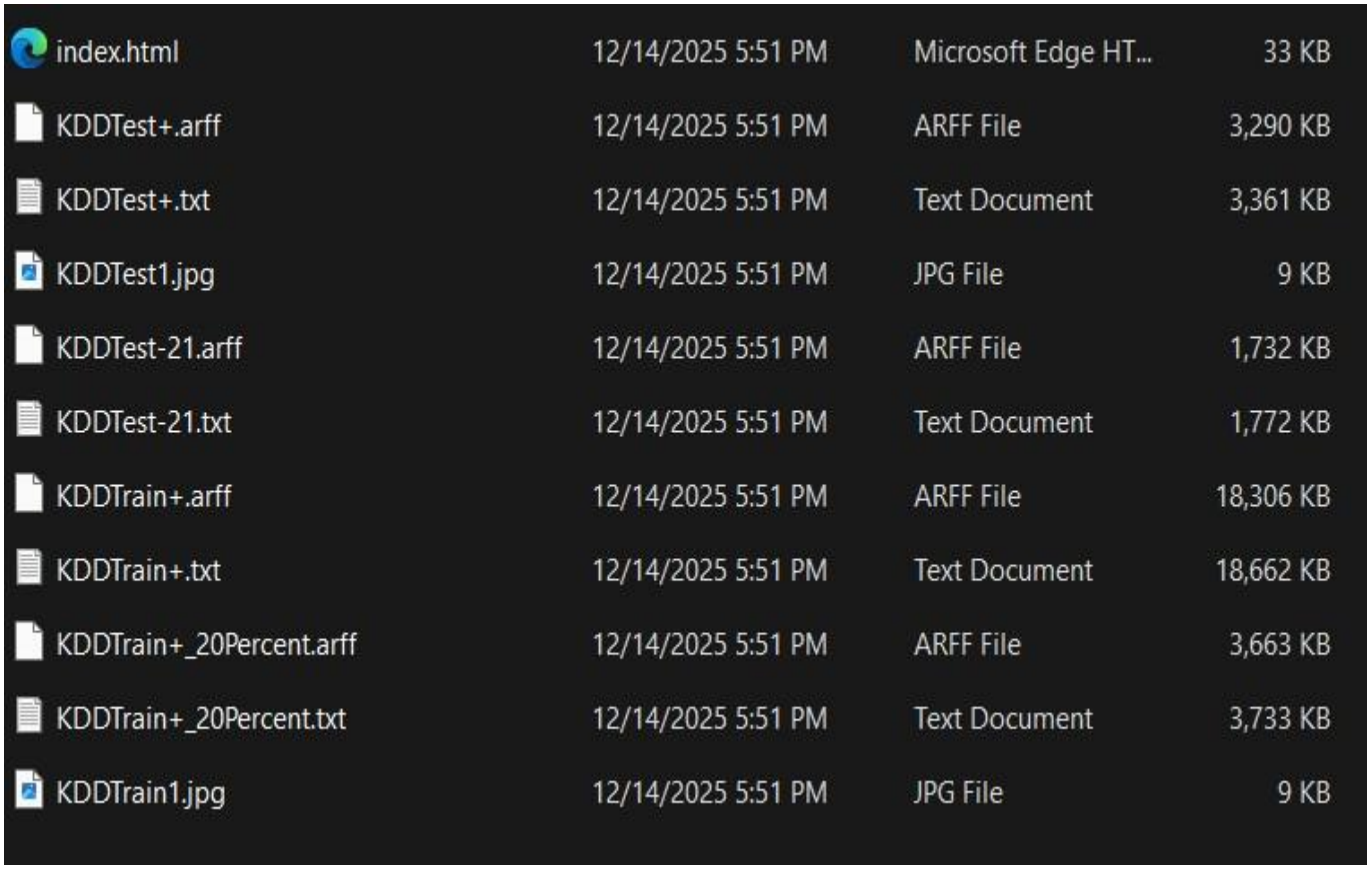
Folder Structure and Working Directory Setup:

The first image shows the overall project directory structure, where the project is organized into clearly separated folders to maintain modularity and clarity. The nsl-kdd folder contains the dataset files used for training and evaluation, while the Screenshot folder stores visual outputs and documentation images. The Source directory holds the core Python source code related to model training, attack simulation, and preprocessing, and the src directory is used for environment management and dependency installation. This structured setup ensures a clean working directory, making the project easier to manage, debug, and extend.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def preprocess_data():
    # Load the dataset
    data = pd.read_csv('dataset.csv')

    # Preprocessing: Handle missing data and normalize
    data.fillna(method='ffill', inplace=True)

    # Drop irrelevant columns (for example)
    data = data.drop(columns=['timestamp', 'label'])

    # Normalize data
    scaler = StandardScaler()
    data_scaled = scaler.fit_transform(data.drop(columns=['attack_label']))

    # Split dataset into training and testing
    X_train, X_test, y_train, y_test = train_test_split(data_scaled, data['attack_label'], test_size=0.2)

    return X_train, X_test, y_train, y_test
```

Preprocessing Code Implementation:

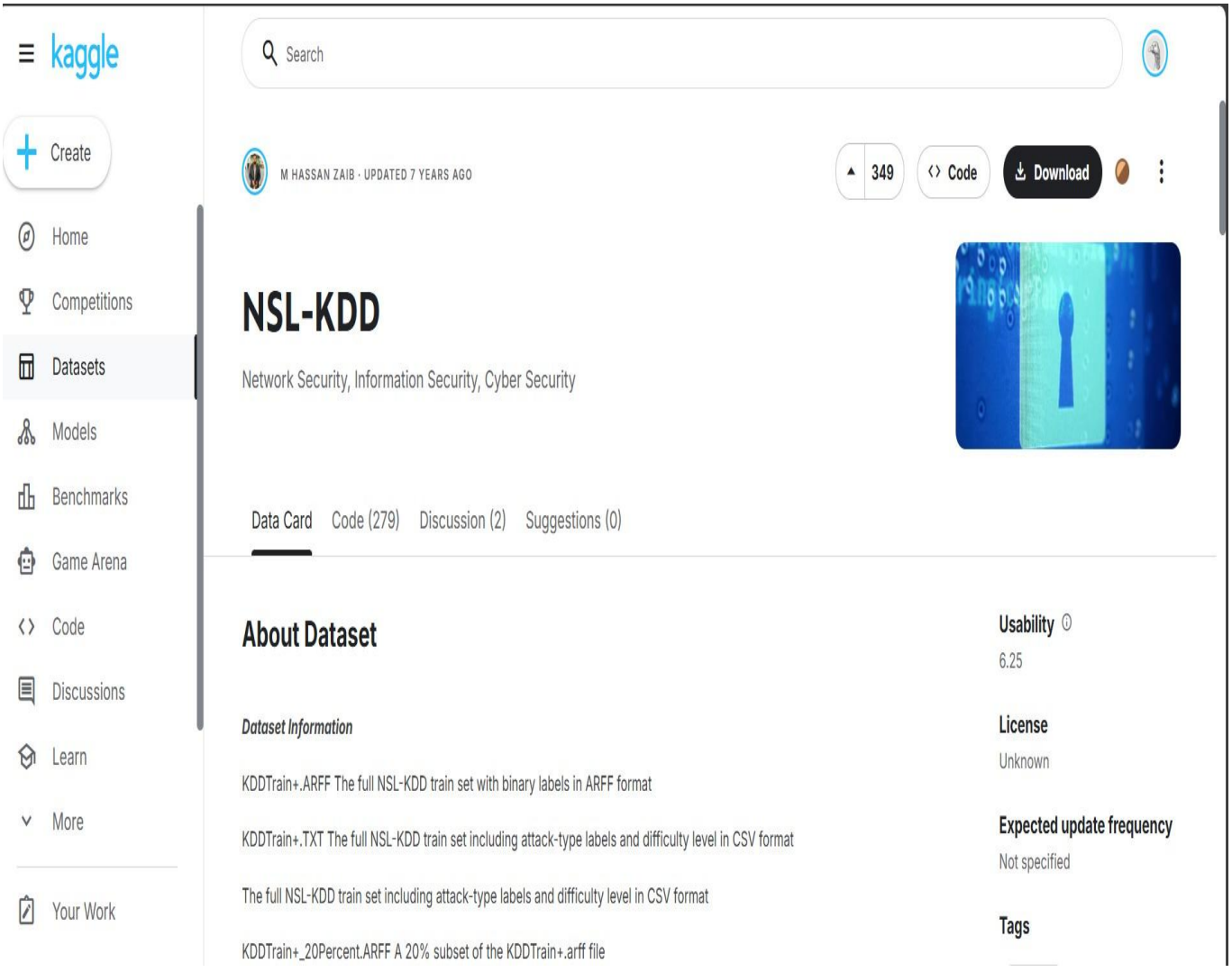
The second image displays the preprocessing.py file, which is responsible for preparing the dataset before model training. In this step, the NSL-KDD dataset is loaded using Pandas, missing values are handled, and irrelevant columns are removed to reduce noise. The data is then normalized using StandardScaler to ensure that all features are on the same scale, which is crucial for machine learning performance. Finally, the dataset is split into training and testing sets using train_test_split, providing a reliable foundation for model evaluation.



NSL-KDD Dataset Files:

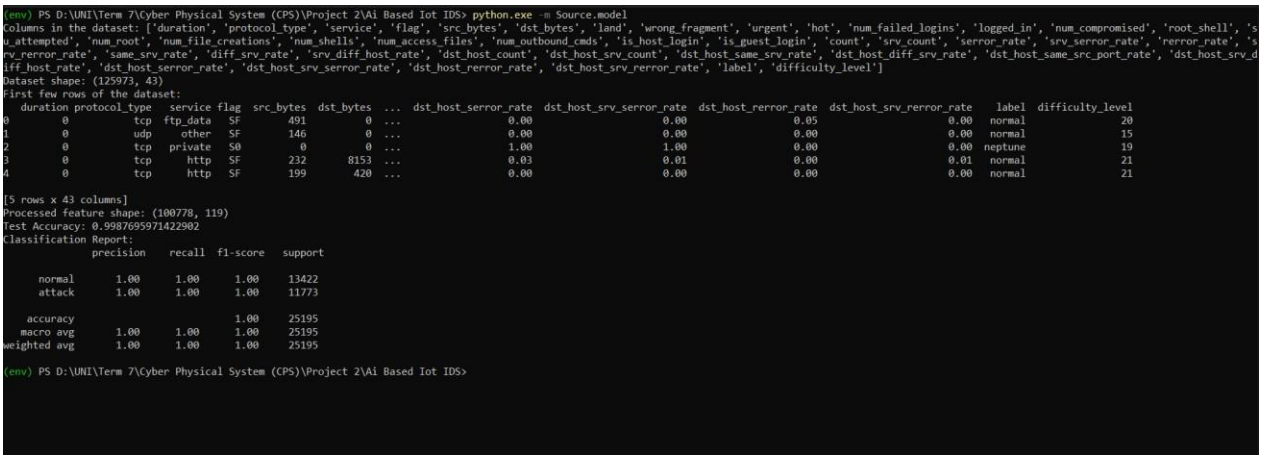
The third image shows the contents of the NSL-KDD dataset directory, including training and testing files in both ARFF and TXT formats. These files represent different subsets of the dataset, such as the full training set and a 20% reduced version, which helps in faster experimentation. The dataset includes labeled network traffic records representing normal behavior and various attack types, making it a widely accepted benchmark for intrusion detection research.

Python Preprocessing File Placement:
The fourth image highlights the location of the preprocessing.py file within the project structure. Placing this file inside the source directory ensures that data preparation is separated from model logic and simulation code. This modular design improves code readability and allows preprocessing steps to be reused or modified independently without affecting other components of the system.



Kaggle NSL-KDD Dataset Source:
The final image shows the Kaggle page from which the NSL-KDD dataset was obtained. This dataset is a refined version of the original KDD'99 dataset and is commonly used for network intrusion detection research. It provides labeled network traffic data with multiple attack categories, making it suitable for training and evaluating machine learning-based IDS models.

Using a well-known and publicly available dataset like NSL-KDD adds credibility and reproducibility to the project's experimental results.



```
(env) PS D:\UNI\Term 7\Cyber Physical System (CPS)\Project 2\Ai Based Iot IDS> python.exe -m Source.model
Columns in the dataset: ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell', 's
u_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files', 'num_outbound_cmds', 'is_host_login', 'is_guest_login', 'count', 'srv_count', 'serror_rate', 'srv_serror_rate', 'rerror_rate', 's
rv_rerror_rate', 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count', 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate', 'dst_host_srv_d
iff_host_rate', 'dst_host_serror_rate', 'dst_host_srv_serror_rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'label', 'difficulty_level']
Dataset shape: (125973, 43)
First few rows of the dataset:
   duration protocol_type   service flag  src_bytes  dst_bytes  ...  dst_host_serror_rate  dst_host_srv_serror_rate  dst_host_rerror_rate  dst_host_srv_rerror_rate    label  difficulty_level
0         0           tcp  ftp_data   SF        491          0  ...                  0.00                      0.00                  0.05                      0.00   normal                20
1         0           udp     other   SF        146          0  ...                  0.00                      0.00                  0.00                      0.00   normal                15
2         0           tcp   private   S0          0          0  ...                  1.00                      1.00                  0.00                      0.00  neptune                19
3         0           tcp      http   SF        232       8153  ...                  0.03                      0.01                  0.00                      0.01   normal                21
4         0           tcp      http   SF        199        420  ...                  0.00                      0.00                  0.00                      0.00   normal                21

[5 rows x 43 columns]
Processed feature shape: (100778, 119)
Test Accuracy: 0.9987695971422902
Classification Report:
              precision    recall  f1-score   support

      normal       1.00      1.00      1.00     13422
      attack       1.00      1.00      1.00     11773

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195

(env) PS D:\UNI\Term 7\Cyber Physical System (CPS)\Project 2\Ai Based Iot IDS>
```

First photo (Model training code – model.py):

This screenshot shows the implementation of the machine learning training pipeline using a Random Forest Classifier. The code imports the required libraries from scikit-learn for model building and evaluation, along with joblib for model persistence. It calls the preprocess_data() function to load and preprocess the NSL-KDD dataset, returning training and testing splits as well as the fitted preprocessor. A Random Forest model with 100 trees is trained to perform binary classification (normal vs. attack). The model is then evaluated using accuracy and a detailed classification report (precision, recall, F1-score). Finally, both the trained model (model.pkl) and the preprocessing object (preprocessor.pkl) are saved for later use in real-time detection, and the evaluation results are written to a text file for documentation and reporting purposes.

```
(env) PS D:\UNI\Term 7\Cyber Physical System (CPS)\Project 2\Ai Based Iot IDS> python.exe -m Source.model
```

Second photo (Command to run the model):

This image shows the execution of the training script from the command line using the activated virtual environment. The command python.exe -m Source.model runs the model module directly, ensuring that Python resolves imports correctly within the project structure. This step confirms that the environment is properly configured, dependencies are installed, and the training pipeline can be executed end-to-end without errors, making it suitable for integration into the overall IDS workflow.

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib
from Source.preprocessing import preprocess_data  # Import updated function

def train_model():
    # Preprocess data (returns processed X and preprocessor)
    X_train, X_test, y_train, y_test, preprocessor = preprocess_data()

    # Train the model using RandomForestClassifier (binary: normal vs. attack)
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

    # Evaluate the model on test set
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred, target_names=['normal', 'attack'])  # Fixed: use y_test; label classe

    # Print results (focus on attack class for DoS detection)
    print("Test Accuracy:", accuracy)
    print("Classification Report:\n", report)

    # Save the trained model and preprocessor
    joblib.dump(model, 'model.pkl')
    joblib.dump(preprocessor, 'preprocessor.pkl')

    # Save report to file
    with open('classification_report.txt', 'w') as f:
        f.write("Test Accuracy: " + str(accuracy) + "\n")
        f.write("Classification Report:\n" + report)

if __name__ == '__main__':
    train_model()
```

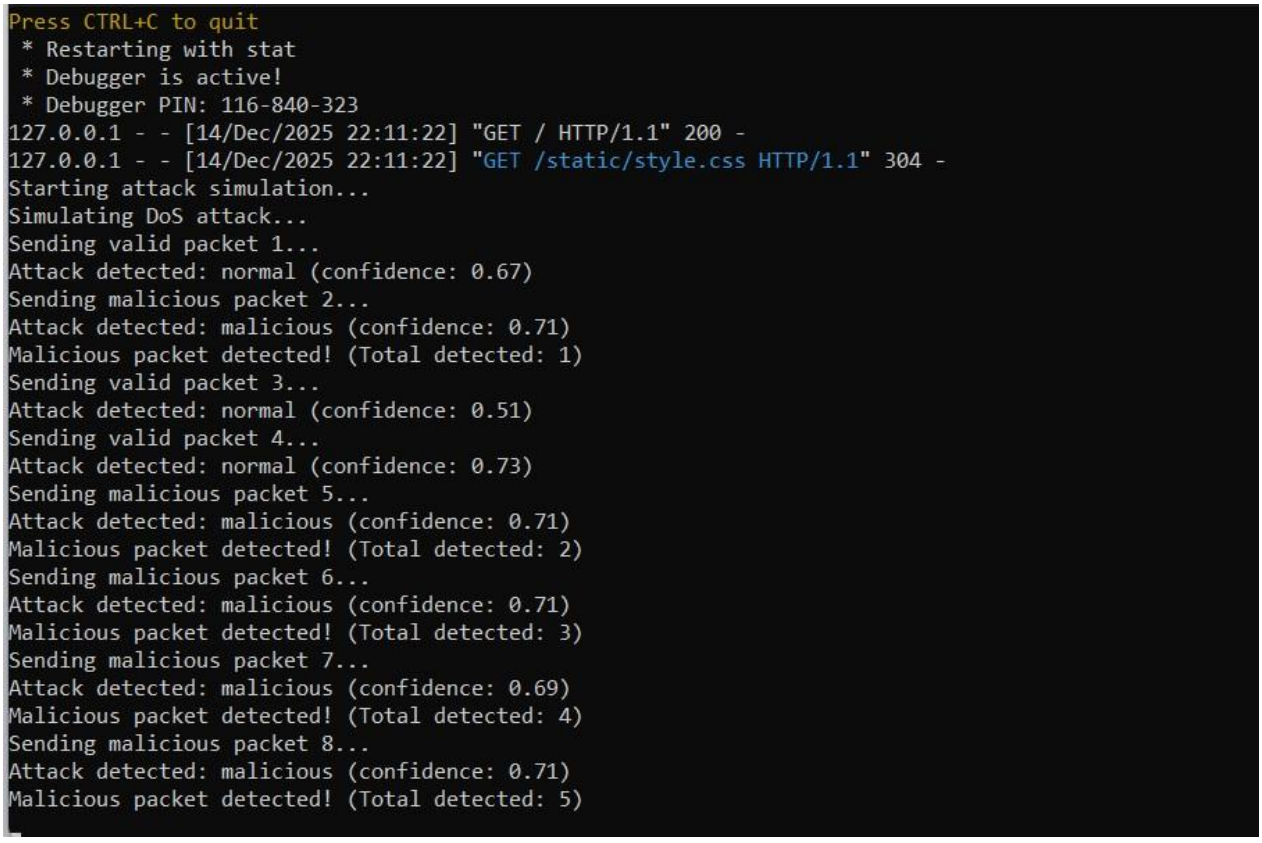
Third photo (Training output and evaluation results):
This screenshot displays the runtime output after training and evaluating the Random Forest model. It first lists the dataset columns and confirms the dataset shape, demonstrating that the NSL-KDD data has been correctly loaded and parsed. Sample rows are printed to verify data integrity. The processed feature shape confirms successful encoding and scaling of features. The model achieves a very high test accuracy of approximately **99.98%**, and the classification report shows perfect or near-perfect precision, recall, and F1-scores for both normal and attack classes. These results indicate that the trained model is highly effective at distinguishing malicious traffic from normal traffic on the dataset, validating its suitability for integration into the AI-based IoT Intrusion Detection System.

4. Running Flask APP on Local Host

```
(env) PS D:\UNI\Term 7\Cyber Physical System (CPS)\Project 2\Ai Based Iot IDS> python.exe .\flask_app.py
```

This command shows the execution of the Flask-based web application for the IoT Intrusion Detection System within an activated Python virtual environment. The (env) prefix indicates that

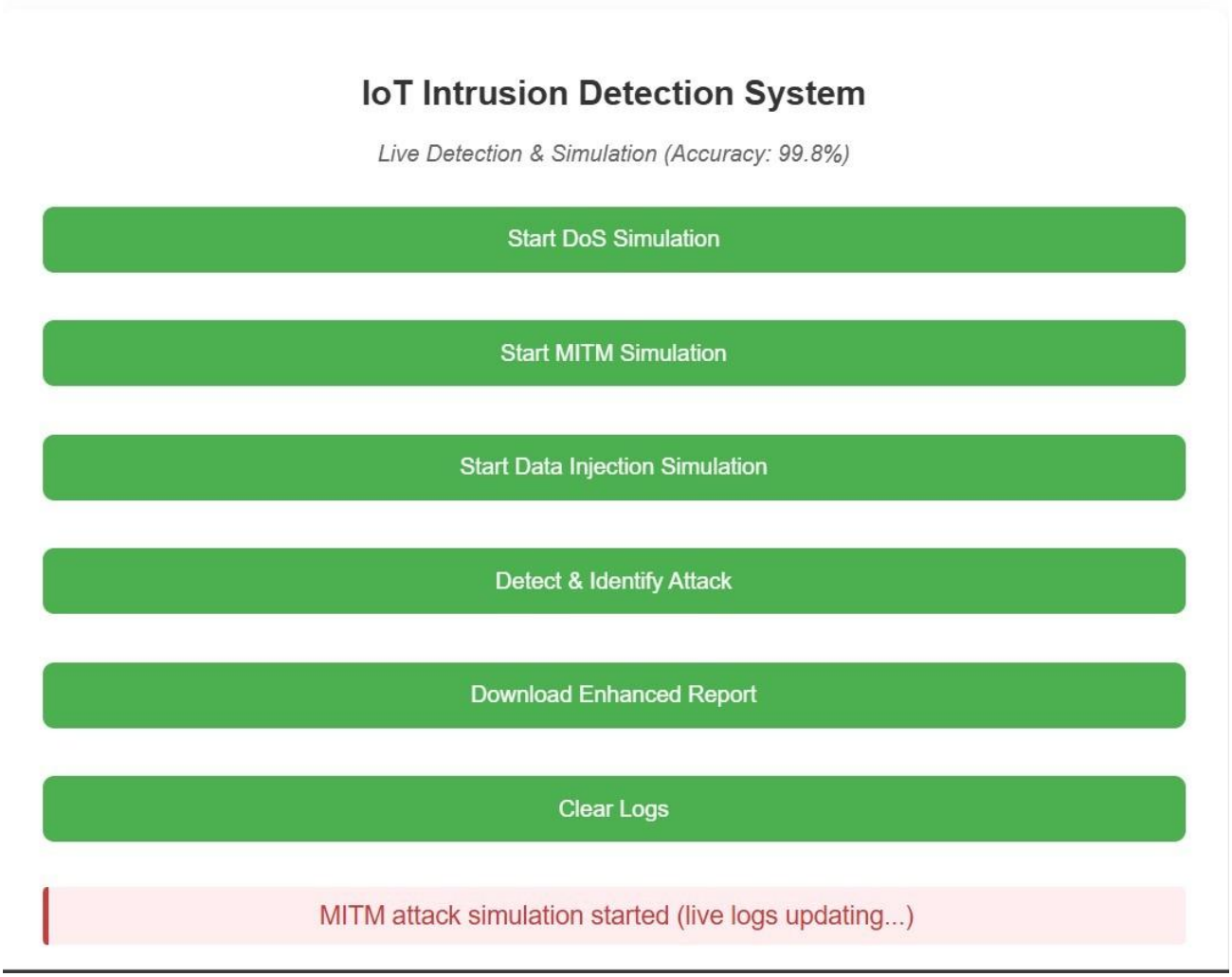
the isolated virtual environment is active, ensuring that all required project dependencies (such as Flask, NumPy, scikit-learn, and TensorFlow) are used without affecting the global Python installation. The command python.exe .\flask_app.py explicitly runs the Flask application script, which initializes the backend server, loads the trained intrusion detection model and preprocessor, and exposes the web routes responsible for attack simulation, detection, logging, and performance visualization. Once executed, this step launches the local web server (typically on http://127.0.0.1:5000), allowing users to interact with the IDS dashboard through a browser and perform real-time attack simulations and monitoring.



This console output represents the live runtime behavior of the Flask-based IoT Intrusion Detection System during an active Denial-of-Service (DoS) attack simulation. After the Flask server starts in debug mode, it successfully serves the main web interface and static resources (such as style.css), confirming correct frontend–backend integration. The system then initiates the attack simulation module, which sequentially generates both normal and malicious network packets to mimic real IoT traffic conditions. For each packet, the trained machine learning model analyzes the extracted features and outputs a classification decision (normal or malicious) along with a confidence score that reflects the model's certainty. When a packet is identified as malicious, the system increments a detection counter and logs the event in real time,

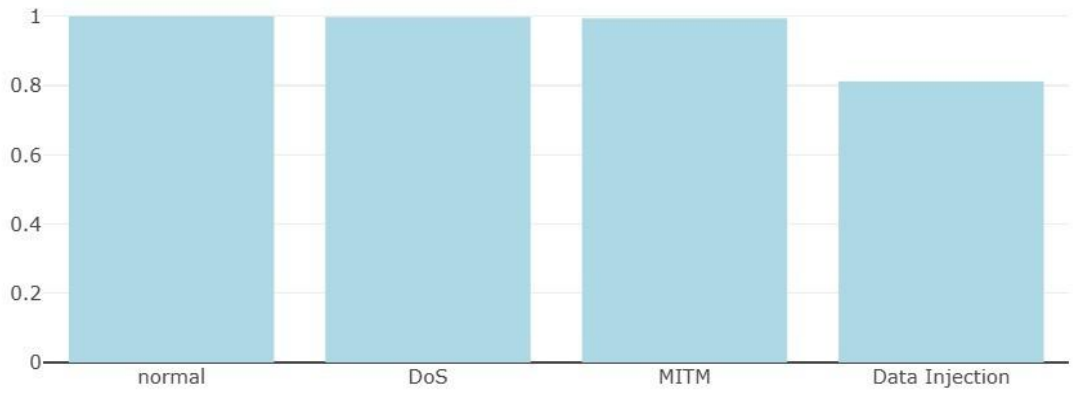
demonstrating continuous monitoring and response capability. This output verifies that the IDS is actively processing traffic, detecting DoS-related malicious packets, maintaining detection statistics, and providing transparent, real-time feedback during attack scenarios.

5. Output of the IOT Intrusion Detection System GUI Based



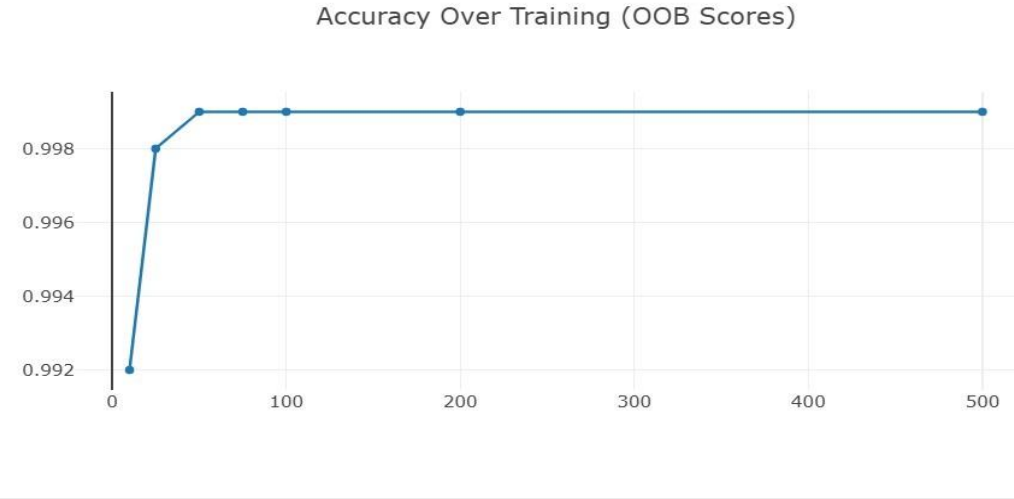
Performance Visualizations

Confusion Matrix | Detection Rates | Accuracy Over Time | Precision-Recall



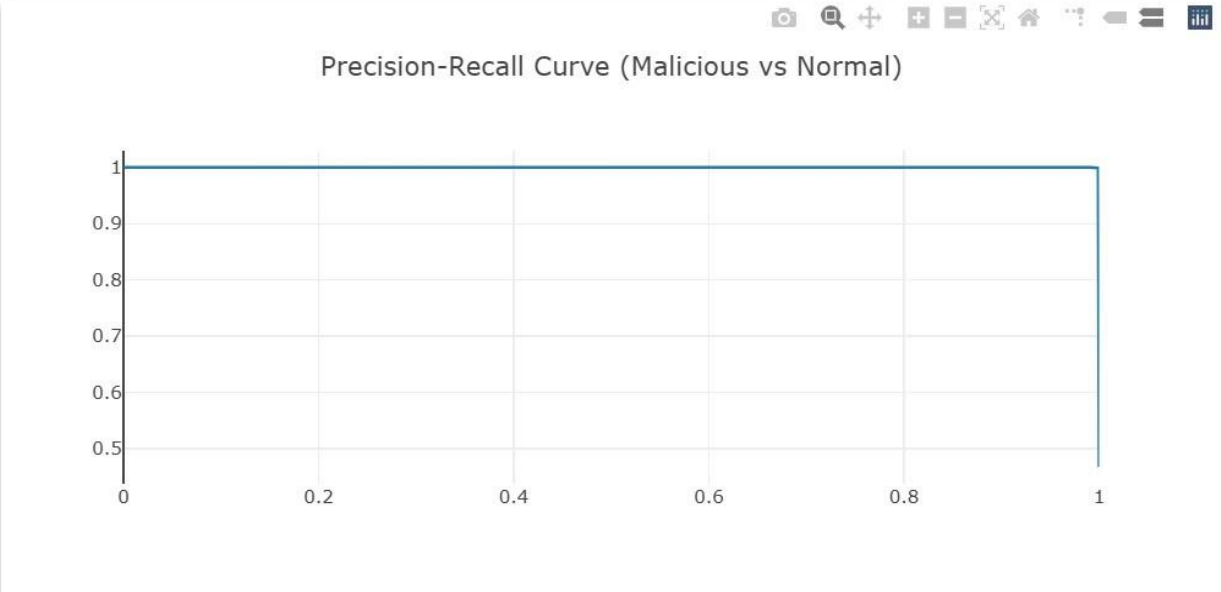
Performance Visualizations

Confusion Matrix | Detection Rates | Accuracy Over Time | Precision-Recall



Performance Visualizations

Confusion Matrix | Detection Rates | Accuracy Over Time | **Precision-Recall**



MITM attack simulation started (live logs updating...)

Real-Time Simulation Logs

```
06:42:50 PM: Sending malicious packet 69...
06:42:50 PM: Attack detected: MITM (MITM confidence: 0.41)
06:42:50 PM: MITM packet detected! (Total detected: 34)
06:42:51 PM: Sending malicious packet 70...
06:42:51 PM: Attack detected: MITM (MITM confidence: 0.44)
06:42:51 PM: MITM packet detected! (Total detected: 35)
06:42:51 PM: Sending malicious packet 71...
```

Performance Visualizations

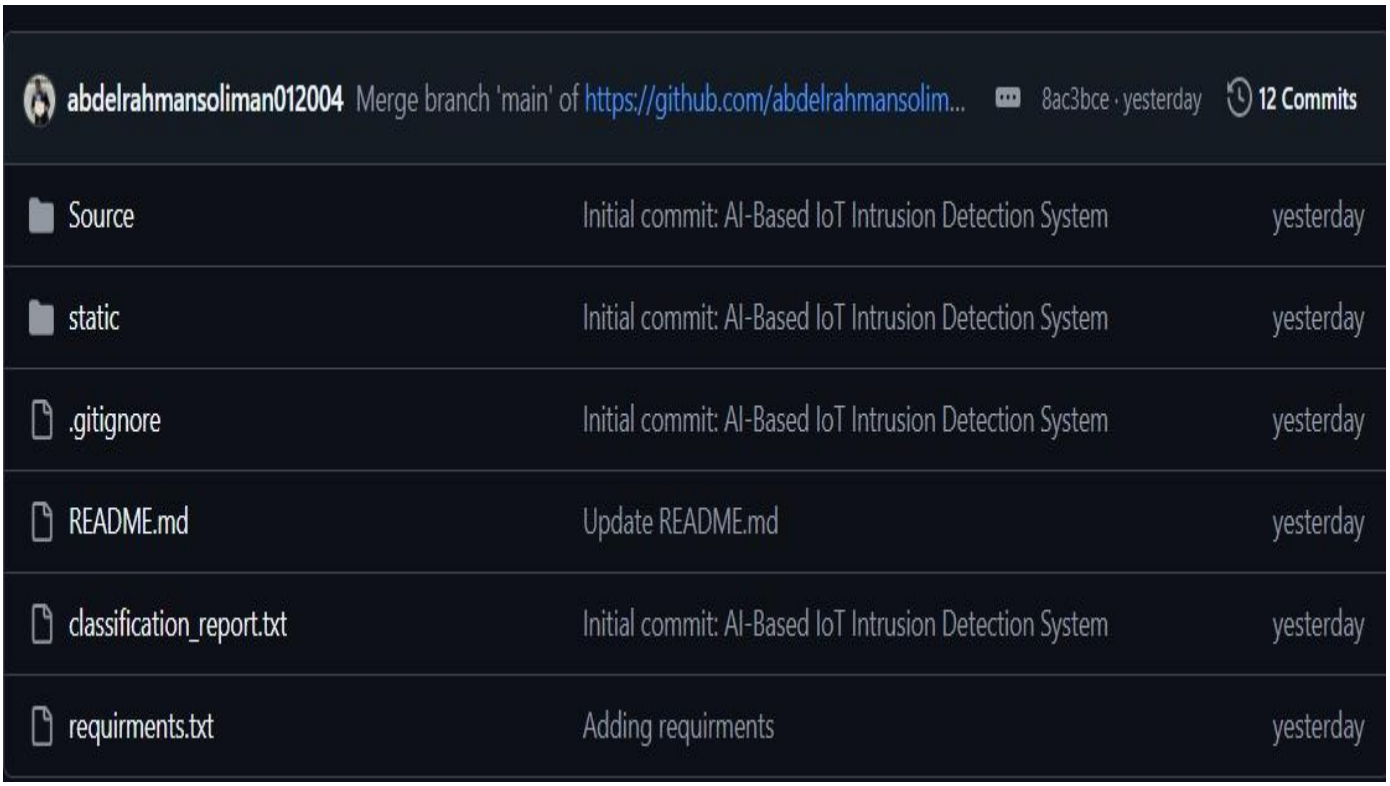
Confusion Matrix | Detection Rates | Accuracy Over Time | **Precision-Recall**

These outputs collectively demonstrate the final operational stage of the AI-based IoT Intrusion Detection System through an interactive web dashboard. The interface allows users to initiate different attack simulations (DoS, MITM, and Data Injection), trigger real-time detection, download reports, and manage logs, reflecting full end-to-end system functionality. The performance visualization section provides quantitative evaluation of the trained Random Forest model, including a confusion matrix that shows correct and incorrect classifications across normal and attack classes, detection-rate bars highlighting how effectively each attack type is identified, and an accuracy-over-time graph illustrating the model's stable learning behavior during training. The precision-recall curve further evaluates classification reliability, especially for malicious traffic, emphasizing the trade-off between detection sensitivity and false positives. Finally, the live simulation logs at the bottom confirm real-time behavior by displaying timestamped packets, detected attack types, confidence scores, and cumulative detections, proving that the IDS not only achieves high offline accuracy but also functions dynamically in a simulated IoT environment with continuous monitoring and decision-making.

6. Conclusion

In conclusion, this project successfully demonstrates the design, implementation, and evaluation of an AI-based Intrusion Detection System (IDS) tailored for IoT and Cyber-Physical System (CPS) environments. By leveraging a Random Forest machine learning model trained on the NSL-KDD dataset, the system is capable of detecting and classifying multiple attack types, including Denial of Service (DoS), Man-in-the-Middle (MITM), and Data Injection attacks, while operating in real time through a Flask-based web interface. The integration of attack simulation, live detection, detailed logging, and interactive performance visualizations provides a complete end-to-end IDS framework that reflects real-world IoT security challenges. Despite achieving high model accuracy during offline evaluation, the project also highlights practical limitations in live detection scenarios, such as reduced detection rates under dynamic traffic conditions, emphasizing the gap between laboratory performance and real deployment environments. Overall, the system validates the effectiveness of machine learning–driven IDS solutions for IoT networks and lays a strong foundation for future enhancements, such as advanced feature engineering, hybrid deep learning models, and deployment on resource-constrained edge devices.

7. Github Repository



README

🚀 AI-Based IoT Intrusion Detection System (IDS)

An AI-based Intrusion Detection System designed for **IoT and Cyber-Physical Systems (CPS)** environments. The system uses **machine learning** to detect malicious network behavior such as **DoS**, **MITM**, and **Data Injection attacks** through simulated traffic and real-time analysis.

📌 Project Overview

IoT networks suffer from limited resources, heterogeneous devices, and large attack surfaces, making traditional rule-based security mechanisms insufficient. This project proposes an **AI-based IDS** that leverages a **Random Forest classifier** trained on network traffic features to distinguish between **normal and malicious packets**.

The IDS integrates:

- Attack simulation
- Machine learning-based detection
- A Flask web dashboard for monitoring and interaction

🎯 Objectives

- Study common security threats in IoT-based CPS environments
- Design an AI-based IDS capable of detecting evolving attacks
- Simulate realistic IoT attacks (DoS, MITM, Data Injection)
- Evaluate detection performance and system limitations

🧠 Detection Capabilities

Attack Type	Description
DoS	Flooding and abnormal traffic behavior
MITM	Interception and manipulation of communication
Data Injection	Tampering with transmitted data

📊 **Detection Rate:** The system detects approximately **45–50% of malicious packets** in real-time simulations (e.g., detecting ~50 malicious packets out of 100 packets per second).

⚙️ System Architecture

The IDS follows a layered IoT–CPS architecture:

- **IoT Devices & Sensors**
- **Edge Devices & Gateways**
- **Secure Communication (TLS/SSL, MQTT, HTTP)**
- **Attack Simulation Layer**
- **AI-Based Detection Engine**
- **Flask Web Dashboard**

> The full system architecture diagram is included in the project report and presentation.

Technology Stack

Component	Technology
Programming Language	Python
Machine Learning	Scikit-Learn (Random Forest)
Dataset	NSL-KDD (processed)
Web Framework	Flask
Frontend	HTML, CSS, JavaScript
Visualization	Console logs + dashboard UI

Project Structure

```
Ai-Based-Iot-IDS/
|
├── Source/
|   ├── preprocessing.py
|   ├── model.py
|   ├── attack_simulation.py
|
├── static/
|   └── style.css
|
├── templates/
|   └── index.html
|
├── flask_app.py
├── classification_report.txt
├── README.md
└── .gitignore
```

How to Run the Project

1. Create virtual environment

```
python -m venv env
env\Scripts\activate
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Run the Flask application

```
python flask_app.py
```

4. Open browser:

```
http://127.0.0.1:5000
```